DATABASE-APP

Mini Relational Database Management System in C++

BazePodataka-APP is a console-based relational database management system developed in C++.

The project originally started as a simple first-semester application for storing and manipulating data inside a single predefined relation. Its initial functionality consisted of inserting, removing, searching, sorting, and storing records.

The application has since been completely redesigned and expanded into a small educational **Relational Database Management System (RDBMS)** capable of managing multiple tables with different schemas, relationships between tables, integrity constraints, persistent storage, and relational algebra operations.

The main purpose of this project is not to replace production database systems such as MySQL, PostgreSQL, or SQLite. Instead, the goal is to implement and demonstrate some of the fundamental concepts that exist underneath relational database systems directly in C++.

The application therefore provides a practical connection between:

- C++ programming
- data structures
- relational database theory
- relational algebra
- database schemas
- primary and foreign keys
- referential integrity
- persistent data storage
- operations between multiple relations

Project Evolution

The first version of the application was based on a single predefined structure containing attributes such as a name, price, year, and quality.

The program supported several basic operations:

1. Insert records
2. Remove records
3. Search records
4. Sort records
5. Store records
6. Exit the application

This approach was useful for learning basic data manipulation, but it had an important limitation: the program could effectively manage only one predefined type of relation.

The current version completely remodels this architecture.

Instead of representing the entire database through one fixed structure, the application now uses a generic relational model:

```
Database
   |
   +-- Table
   |     |
   |     +-- Columns
   |     |
   |     +-- Rows
   |     |     |
   |     |     +-- Values
   |     |
   |     +-- Primary Key
   |     |
   |     +-- Foreign Keys
   |
   +-- Table
   |
   +-- Table
   |
   +-- ...
```

This allows multiple tables with completely different schemas to exist inside the same database.

Core Architecture

The database is represented by several fundamental C++ structures.

Row

A row represents one tuple inside a relation.

Conceptually:

```
struct Red {
    string vrijednosti[MAX_STUPACA];
};
```

Each row contains values corresponding to the columns of its table.

Table

A table represents a relation inside the database.

Each table contains:

- table name
- column names
- number of columns
- rows
- number of rows
- primary key definition
- foreign key definitions

Conceptually:

```
Table
 |
 +-- Name
 +-- Columns
 +-- Rows
 +-- Primary Key
 +-- Foreign Keys
```

Because columns are no longer hardcoded into one specific structure, different tables can represent completely different entities.

For example:

```
Manufacturers
--------------------------------
id | name       | country
1  | Volkswagen | Germany
2  | BMW        | Germany
3  | Toyota     | Japan
```

while another table may contain:

```
Cars
-----------------------------------------------
id | model   | manufacturer_id | price
1  | Golf    | 1               | 18000
2  | Passat  | 1               | 26000
3  | 320i    | 2               | 35000
4  | Corolla | 3               | 22000
```

These tables have different schemas but can coexist inside the same database.

Database

The database represents the highest level of the system.

Conceptually:

```
struct BazaPodataka {
    string naziv;
    Tablica tablice[MAX_TABLICA];
    int brojTablica;
};
```

A database therefore contains multiple tables which can be independently managed and connected through relationships.

Main Functionalities

The current version of BazePodataka-APP provides the following main functionality:

1. Display all tables
2. Create tables
3. Delete tables
4. Insert rows
5. Delete rows
6. Display table contents
7. Define foreign keys
8. Execute relational algebra operations
9. Save the database
10. Load an existing database
11. Exit the application safely

Each functionality is implemented through dedicated functions while the console menus provide interaction between the user and the database engine.

Table Management

Creating Tables

The application allows users to dynamically create new tables.

When creating a table, the user defines:

- table name
- number of columns
- column names
- optional primary key

This means the schema is not permanently hardcoded into the application.

For example, a user can create:

```
Students

id
first_name
last_name
year
```

and later create:

```
Courses

id
name
ects
```

without modifying the C++ source code.

Deleting Tables

Existing tables can be removed from the database.

Before deleting a table, the application checks whether another table contains a foreign key referencing it.

If such a relationship exists, deletion is rejected in order to preserve referential integrity.

For example:

```
Cars.manufacturer_id
        |
        v
Manufacturers.id
```

If `Cars` contains rows referencing `Manufacturers`, the referenced table cannot simply be removed while the relationship exists.

Row Management

Inserting Rows

Rows can be dynamically inserted into any existing table.

The application requests a value for every column defined in the table schema.

Before the row is inserted, several integrity checks may be performed.

If the table contains a primary key, the application verifies that:

- the primary key value is not empty
- the primary key value is unique

If the table contains foreign keys, the application verifies that the referenced values actually exist in the corresponding tables.

Only rows satisfying these constraints are inserted.

Removing Rows

Rows can be removed using their primary key.

Before deleting a row, the system checks whether its primary key is currently referenced by a foreign key from another table.

If the row is referenced, deletion is rejected.

This prevents the creation of invalid references between tables.

Primary Keys

The application supports basic **PRIMARY KEY** constraints.

A primary key uniquely identifies a tuple inside a relation.

For example:

```
Manufacturers

id [PRIMARY KEY]
name
country
```

Valid data:

```
1 | Volkswagen | Germany
2 | BMW        | Germany
3 | Toyota     | Japan
```

The application prevents duplicate primary key values.

Therefore, inserting another manufacturer with:

```
id = 1
```

would be rejected.

Primary key values are also required to contain a value.

Foreign Keys

The application supports **FOREIGN KEY** relationships between tables.

A foreign key connects an attribute from one table with the primary key of another table.

Example:

```
Cars.manufacturer_id
        |
        | FOREIGN KEY
        v
Manufacturers.id
```

This creates a relationship between cars and their manufacturers.

The application verifies that foreign key values reference existing values.

For example, assume the following manufacturers exist:

```
1 | Volkswagen
2 | BMW
3 | Toyota
```

The following car is valid:

```
10 | Golf | 1 | 18000
```

because manufacturer `1` exists.

However:

```
11 | ExampleCar | 99 | 20000
```

would be rejected if manufacturer 99 does not exist.

Referential Integrity

Referential integrity is one of the central concepts implemented in the application.

The system attempts to prevent invalid relationships between tables.

For example:

```
Manufacturers
---------------
id
1
2
3

Cars
---------------------------
id | manufacturer_id
1  | 1
2  | 1
3  | 2
```

Manufacturer 1 cannot be deleted while cars still reference that value.

Without this protection, the database could contain:

```
Cars.manufacturer_id = 1
```

while:

```
Manufacturers.id = 1
```

no longer exists.

The application prevents this situation.

Relational Algebra

One of the most important additions to the current version is the implementation of **relational algebra**.

Relational algebra provides the theoretical foundation for manipulating relations in relational database systems.

The application currently implements:

```
Selection           sigma
Projection          pi
Union               union
Intersection        intersection
Difference          difference
Cartesian Product   product
Join                join
```

Conceptually:

```
σ  Selection
π  Projection
∪  Union
∩  Intersection
−  Difference
×  Cartesian Product
⋈  Join
```

Selection

Selection chooses tuples satisfying a specified condition.

Conceptually:

```
σ condition (Relation)
```

Example:

```
σ price > 20000 (Cars)
```

Given:

```
id | model   | price
1  | Golf    | 18000
2  | Passat  | 26000
```

```
3  | 320i    | 35000
4  | Corolla | 22000
```

the resulting relation contains:

```
2 | Passat  | 26000
3 | 320i    | 35000
4 | Corolla | 22000
```

The implementation supports comparison operators such as:

```
=
==
!=
>
<
>=
<=
```

The program attempts to perform numerical comparison when both operands represent numbers and otherwise performs string comparison.

Projection

Projection selects specific attributes from a relation.

Conceptually:

```
π attribute1, attribute2 (Relation)
```

Example:

```
π model, price (Cars)
```

The original relation:

```
id | model   | manufacturer_id | price
1  | Golf    | 1               | 18000
2  | Passat  | 1               | 26000
3  | 320i    | 2               | 35000
```

can therefore be transformed into:

```
model  | price
Golf   | 18000
Passat | 26000
320i   | 35000
```

Union

Union combines tuples from two compatible relations.

Conceptually:

```
R ∪ S
```

For union to be executed, the relations must be compatible.

The current implementation verifies that the relations contain the same number of columns and corresponding column names.

Duplicate tuples are not intentionally added to the result.

Intersection

Intersection returns tuples existing in both compatible relations.

Conceptually:

```
R ∩ S
```

Only tuples present in both relations become part of the resulting relation.

Difference

Difference returns tuples that exist in the first relation but do not exist in the second relation.

Conceptually:

```
R - S
```

For example:

```
R = {A, B, C}
S = {B, C}
```

produces:

```
R - S = {A}
```

The two relations must be compatible.

Cartesian Product

The Cartesian product combines every tuple from one relation with every tuple from another relation.

Conceptually:

```
R × S
```

If relation `R` contains:

```
m
```

rows and relation `S` contains:

```
n
```

rows, the Cartesian product can contain:

```
m × n
```

rows.

The resulting table contains attributes from both source relations.

Column names are qualified using their table names to make their origin clear.

Example:

```
Cars.id
Cars.model
Manufacturers.id
Manufacturers.name
```

JOIN

JOIN is one of the most important multi-table operations implemented by the application.

The current implementation performs an equality-based join between two selected attributes.

Conceptually:

```
R ⋈ S
```

For example:

```
Cars ⋈ Manufacturers
```

using:

```
Cars.manufacturer_id = Manufacturers.id
```

Given:

```
Cars

id | model   | manufacturer_id
1  | Golf    | 1
2  | 320i    | 2
3  | Corolla | 3
```

and:

```
Manufacturers

id | name
1  | Volkswagen
2  | BMW
3  | Toyota
```

the JOIN operation can produce:

```
Cars.id | Cars.model | Cars.manufacturer_id | Manufacturers.id |
Manufacturers.name

1 | Golf    | 1 | 1 | Volkswagen
2 | 320i    | 2 | 2 | BMW
3 | Corolla | 3 | 3 | Toyota
```

This functionality allows meaningful relationships between multiple tables to be queried directly inside the C++ application.

Relational Algebra Results

Relational algebra operations do not only print information to the console.

The result itself is represented as another table.

This means a result can optionally be stored inside the database as a new relation.

Conceptually:

```
Cars
   |
   | Selection
   v
ExpensiveCars
```

or:

```
Cars + Manufacturers
          |
          | JOIN
          v
CarsWithManufacturers
```

This makes the relational algebra implementation significantly more flexible because generated relations can become part of the database.

Persistent Storage

The application supports persistent storage using text files.

The database schema is stored in:

```
schema.txt
```

while individual tables are stored in separate text files.

For example:

```
project/
|
+-- BazaPodataka.cpp
+-- schema.txt
+-- Cars.txt
+-- Manufacturers.txt
+-- Students.txt
+-- Courses.txt
```

The exact table files depend on the tables created by the user.

Schema Storage

`schema.txt` stores structural information required to reconstruct the database.

This includes information such as:

- database name
- number of tables
- table names
- number of columns
- column names
- primary key index
- number of foreign keys
- foreign key definitions

Because the schema is stored separately from table data, the application can reconstruct the structure of the database when it is started again.

Table Storage

Each table is stored inside its own `.txt` file.

For example:

```
Cars.txt
```

could contain:

```
id;model;manufacturer_id;price
1;Golf;1;18000
2;Passat;1;26000
3;320i;2;35000
4;Corolla;3;22000
```

The first line represents the table header.

The remaining lines represent tuples.

Values are separated using:

```
;
```

This provides a simple human-readable persistence format.

Database Loading

When the application starts, it attempts to load an existing database from:

```
schema.txt
```

If a valid schema exists, the database structure and corresponding table files are loaded.

If no existing database is found, the user can create a new database.

This allows data to remain available between separate executions of the program.

Main Application Menu

The current application exposes the following main menu:

```
========================================
          MINI RELATIONAL DBMS
========================================

1. Display tables
2. Create table
3. Delete table
4. Add row
5. Delete row
6. Display table contents
7. Add FOREIGN KEY
8. Relational algebra
9. Save database
10. Exit

========================================
```

The relational algebra subsystem contains its own menu:

```
========================================
          RELATIONAL ALGEBRA
========================================

1. Selection
2. Projection
3. Union
4. Intersection
5. Difference
6. Cartesian Product
7. JOIN
8. Back

========================================
```

Example Database

A simple database can contain two related tables.

Manufacturers

```
id | name       | country
---------------------------
1  | Volkswagen | Germany
```

```
2  | BMW          | Germany
3  | Toyota       | Japan
```

Primary key:

```
Manufacturers.id
```

Cars

```
id | model    | manufacturer_id | price
---------------------------------------
1  | Golf     | 1               | 18000
2  | Passat   | 1               | 26000
3  | 320i     | 2               | 35000
4  | Corolla  | 3               | 22000
```

Primary key:

```
Cars.id
```

Foreign key:

```
Cars.manufacturer_id
        |
        v
Manufacturers.id
```

The database can then execute operations such as:

```
σ price > 20000 (Cars)
```

```
π model, price (Cars)
```

and:

```
Cars ⋈ Manufacturers
ON Cars.manufacturer_id = Manufacturers.id
```

This example demonstrates how table management, keys, referential integrity, and relational algebra work together inside the application.

Data Integrity

Several validation mechanisms are implemented to reduce invalid database states.

The application currently checks:

- maximum number of tables
- maximum number of columns
- maximum number of rows
- duplicate table names
- duplicate column names during table creation
- existence of selected tables
- existence of selected columns
- primary key uniqueness
- non-empty primary key values
- foreign key target existence
- foreign key references to primary keys
- existence of referenced values
- referential integrity before row deletion
- referential integrity before table deletion
- relation compatibility for set operations

These checks provide a simplified implementation of constraints normally handled by a complete DBMS.

Current Project Structure

The logical structure of the application can be represented as:

```
User
 |
 v
Console Interface
 |
 v
Main Menu
 |
 +---------------------------+
 |                           |
 v                           v
Database Management     Relational Algebra
 |                           |
 +-- Tables                  +-- Selection
 +-- Rows                    +-- Projection
```

```
 +-- Primary Keys             +-- Union
 +-- Foreign Keys             +-- Intersection
 +-- Referential Integrity    +-- Difference
 |                            +-- Cartesian Product
 |                            +-- JOIN
 v
Database
 |
 +-- Table 1
 +-- Table 2
 +-- Table 3
 |
 v
Persistent Storage
 |
 +-- schema.txt
 +-- Table1.txt
 +-- Table2.txt
 +-- Table3.txt
```

Technical Limits

BazePodataka-APP is an educational relational database implementation rather than a production DBMS.

The current version intentionally uses relatively simple C++ concepts and fixed-size structures.

Several limits are therefore defined through constants such as:

```
MAX_TABLICA
MAX_STUPACA
MAX_REDOVA
MAX_FK
```

The application does **not** currently implement features expected from full database engines, such as:

- SQL parsing
- transactions
- ACID transaction management
- indexes
- query optimization
- concurrency control
- user authentication
- database permissions
- advanced data types
- triggers
- stored procedures
- views

- aggregate functions
- `GROUP BY`
- database networking
- crash recovery
- binary database storage

These are outside the current educational scope of the project.

Why the Project Does Not Use MySQL or SQLite

The application deliberately does not delegate database operations to an existing database engine.

Using MySQL, PostgreSQL, or SQLite would provide significantly more functionality, but the purpose of this project is different.

The project attempts to implement several database concepts directly in C++, including:

```
Table representation
        ↓
Tuple storage
        ↓
Primary keys
        ↓
Foreign keys
        ↓
Referential integrity
        ↓
Relational operations
        ↓
Persistent storage
```

This makes the application useful primarily as a learning project for understanding how relational database concepts can be translated into program logic.

Compilation

The application uses standard C++ libraries such as:

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <algorithm>
#include <limits>
```

The final implementation avoids requiring `<filesystem>` so that it remains compatible with older C++ development environments.

A typical compilation command using `g++` is:

```
g++ BazaPodataka.cpp -o BazaPodataka
```

Then run:

Windows

```
BazaPodataka.exe
```

Linux / macOS

```
./BazaPodataka
```

Depending on the compiler version, enabling an appropriate C++ standard explicitly may be useful:

```
g++ -std=c++11 BazaPodataka.cpp -o BazaPodataka
```

Recommended First Test

To verify the complete system, create the following two tables.

Table 1

```
Manufacturers

id
name
country
```

Set:

```
PRIMARY KEY = id
```

Insert:

```
1 | Volkswagen | Germany
2 | BMW        | Germany
3 | Toyota     | Japan
```

Table 2

```
Cars

id
model
manufacturer_id
price
```

Set:

```
PRIMARY KEY = id
```

Insert the following values after defining the relationship as needed:

```
1 | Golf    | 1 | 18000
2 | Passat  | 1 | 26000
3 | 320i    | 2 | 35000
4 | Corolla | 3 | 22000
```

Define:

```
Cars.manufacturer_id
```

as a foreign key referencing:

```
Manufacturers.id
```

Then test:

```
Selection
Projection
Cartesian Product
JOIN
Save Database
```

```
Exit
Restart
Load Database
```

This tests the most important components of the application together.

Development Roadmap

The original version of this project was considered an early-stage application with significant room for expansion.

The major architectural expansion has now been completed.

The project evolved from:

```
Single predefined relation
        ↓
Basic record management
        ↓
Persistent text storage
        ↓
Relational algebra
        ↓
Generic table representation
        ↓
Multiple tables
        ↓
Primary and foreign keys
        ↓
Referential integrity
        ↓
Multi-table JOIN operations
        ↓
Mini Relational DBMS
```

Possible future development could include:

- SQL-like command parsing
- `UPDATE` functionality
- explicit column data types
- `NOT NULL`
- `UNIQUE`
- default values
- auto-incrementing keys
- additional JOIN types
- aggregate functions
- ordering and grouping

- indexes
- dynamic containers instead of fixed arrays
- improved persistence format
- transaction support
- graphical interface
- web interface
- client-server architecture

These improvements are possible future directions rather than requirements for the current version.

Educational Purpose

This project demonstrates the transition from basic procedural data manipulation toward database system design.

During its development, several important concepts are connected inside one application:

```
C++
 |
 +-- Structures
 +-- Arrays
 +-- Functions
 +-- File I/O
 +-- String processing
 +-- Validation
 |
 v
Database Concepts
 |
 +-- Relations
 +-- Tuples
 +-- Attributes
 +-- Schemas
 +-- Primary Keys
 +-- Foreign Keys
 +-- Referential Integrity
 |
 v
Relational Algebra
 |
 +-- Selection
 +-- Projection
 +-- Union
 +-- Intersection
 +-- Difference
 +-- Cartesian Product
 +-- JOIN
```

The result is a practical demonstration of how theoretical relational database concepts can be represented through algorithms and data structures.

Project Status

Completed

The current version contains the core functionality planned for the project:

- [x] Multi-table database management
- [x] Dynamic table schemas
- [x] Row insertion
- [x] Row deletion
- [x] Table creation
- [x] Table deletion
- [x] Table visualization
- [x] Primary keys
- [x] Foreign keys
- [x] Referential integrity checks
- [x] Persistent database storage
- [x] Database loading
- [x] Selection
- [x] Projection
- [x] Union
- [x] Intersection
- [x] Difference
- [x] Cartesian product
- [x] JOIN
- [x] Saving relational operation results as new tables

Current status: stable educational version / core development completed.

Final Note

BazePodataka-APP began as a small console application created to practice basic database-related operations.

After its redesign, it became a significantly more complete educational relational database system.

Rather than storing only one predefined collection of records, the application now models:

databases, tables, schemas, tuples, relationships, constraints, relational operations, and persistent storage.

The project is intentionally kept understandable at the C++ source-code level so that the internal behavior of the database can be studied rather than hidden behind an existing database framework.

Author

Developed as an educational C++ database project and progressively expanded from a first-semester application into a multi-table relational database management system.